

CODE SPORT



This month's column includes a detailed discussion on a couple of coding questions as well as a few more interview questions.

We have been discussing many interview questions over the last couple of columns. A few of our readers had requested for pointers on how to approach and solve algorithmic questions. So let's take up a couple of problems and discuss the solutions in detail. While these questions may appear simple, the key is to understand the approach for each problem and grasp how we can transfer the learnings from these problems to more difficult problems.

(1) *You are given an integer array of N elements. And you are asked to find the top- K most frequent elements.*

One way to approach any problem is to start with a naïve approach which works, and then refine it to make it more efficient. This allows you to have a baseline against which you can measure any improvement. It also helps you to mentally clarify what the corner cases are that you need to deal with. Once you have the normal and corner test cases identified, you need to mentally walk through any improved solution you come up with, and ensure that it handles all the corner cases. So what could be a naïve approach for the above problem?

There are two parts to this problem, one is to obtain the counts of each of the elements and then to find the elements that account for the top K of these counts. When you think about time and space complexity, you need to keep in mind each of the sub-problems that you have to solve to get to the final solution.

Approach 1 is to examine each array element, one at a time, and walk through the complete array to compute how many times it is recurring. This will lead to $O(N^2)$ time complexity. Once you have the counts of the elements, you can sort the counts to get the topmost K frequent elements. This will have a time complexity of $O(N \log N)$ and so the overall time complexity will be $O(N^2)$.

What are the other alternatives? Approach 2 is to sort the array of integers first. Then all the occurrences of the same integer will come together. You can count the occurrences of each element that can be done in the $O(N)$ time frame. You can then return the topmost K frequent elements. But sorting the array would require $O(N \log N)$ time complexity. What are the other alternatives you can think of? How can you reduce the time complexity of each of the two steps?

Finding the counts of each of the elements in our two approaches so far took $O(N^2)$ and $O(N \log N)$. This step was the major bottleneck for the two approaches we've considered so far. Can we do this in $O(N)$ time complexity? Remember that if we try to reduce the time complexity, we may end up having an increase in space complexity. It is a good practice to keep in mind the trade-off. Again, this is part of the iterative approach, whereby we first improve the time complexity at the cost of increasing space complexity and later address the space complexity as well. For computing the frequency counts in $O(N)$ timeframe, we need to look at auxiliary data structures that require additional space. We can use the generic hashmap/dictionary data structure to use the value of the element as the key and its count as the value. In C++ STL, this translates to using the map data structure. This can bring down the complexity of the first step to $O(N)$.

Once we have the frequency counts for all elements in an array, how do we find the topmost K elements? This can be rephrased as follows. Given an array of N integers, how do you find the maximum K elements without sorting it? One of the data structures that should click in your mind is the heap data structure. It takes $O(N)$ to build a max-heap data structure. Each retrieval/deletion of the max element is $O(\log N)$. Hence, for retrieving the topmost K maximum elements, it will take $O(K * \log N)$ time